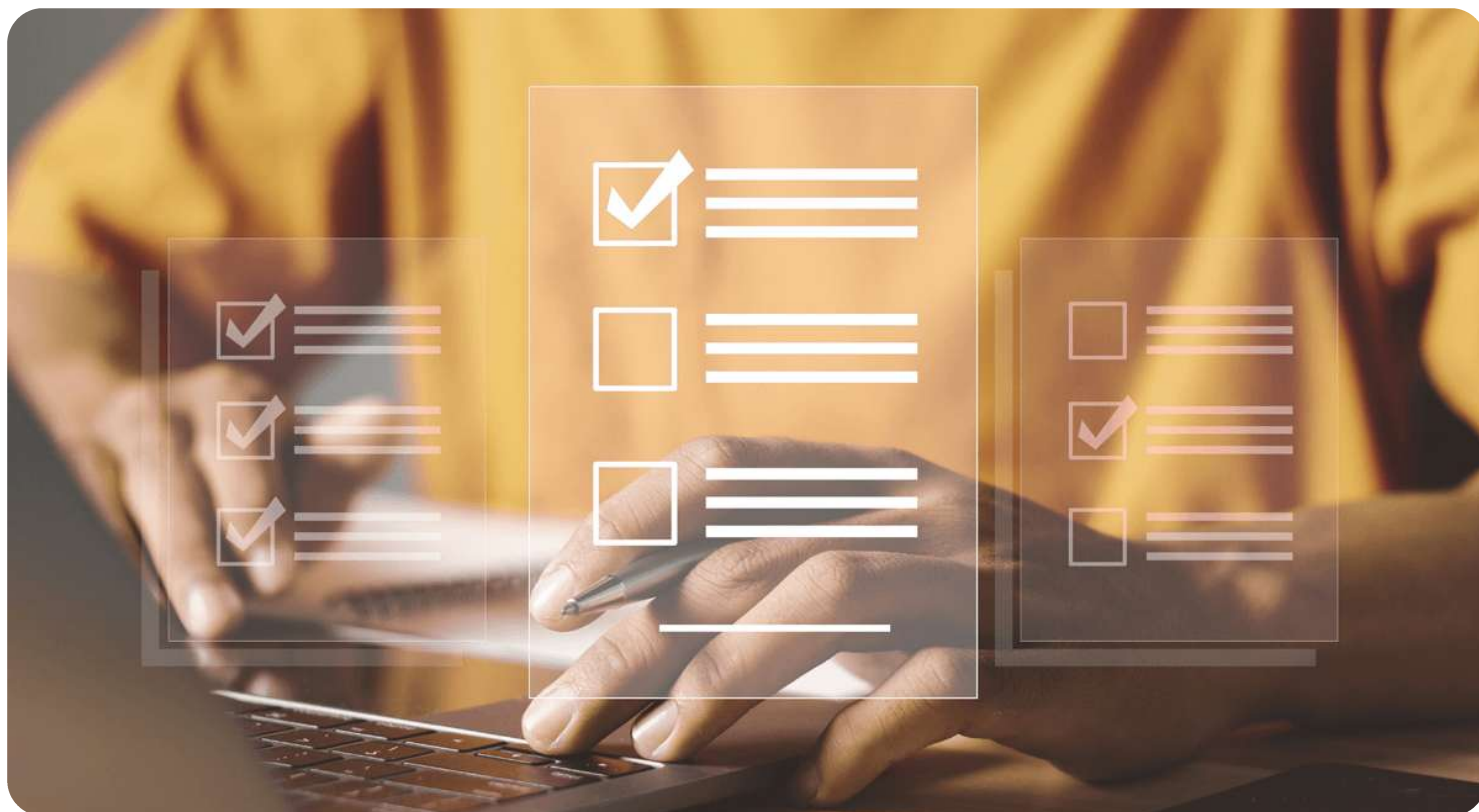




Semana 3

Desarrollo Backend II (PBY2202)

Formato de respuesta

Nombre estudiante:	Rainiero Morroni Migone
Asignatura: Desarrollo Backend II	Carrera: Analista Programador
Profesor: Marcelo Zepeda Aracena	Fecha: 08/06/2026

Entregables

1. Análisis y selección de la estrategia y framework de seguridad:

El proyecto "MINIMARKET PLUS" requiere una modernización de su arquitectura hacia microservicios para gestionar inventarios, ventas en línea y fidelización de clientes. Al exponer estos servicios, el backend enfrenta amenazas críticas que podrían comprometer la integridad del negocio y violar la Ley de Protección de Datos Personales en Chile:

- **Autenticación y Accesos no Autorizados:** Riesgo de que usuarios no autenticados accedan a la gestión de inventario o datos de clientes.
- **Inyección SQL:** Vulnerabilidad donde atacantes podrían manipular consultas a la base de datos (ej. extraer información de tarjetas o modificar precios).
- **XSS (Cross-Site Scripting) y CSRF (Cross-Site Request Forgery):** Riesgo de ejecución de scripts maliciosos o peticiones falsificadas si se manejan sesiones basadas en cookies tradicionales.

Estructura del Control de Acceso Basado en Roles: Con el fin de mitigar eficazmente los riesgos de accesos no autorizados a datos sensibles de operaciones y clientes, se implementó una matriz granular bajo el principio de mínimo privilegio en el framework de seguridad, quedando estructurada de la siguiente manera de acuerdo con los requerimientos del negocio:

- **Cliente (ROLE_USER):** Nivel de seguridad básico. Concede acceso exclusivo a la consulta de catálogos y operaciones de fidelización digital de usuarios.
- **Empleado (ROLE_EMPLOYEE):** Nivel de seguridad medio. Permite la ejecución de tareas operativas críticas de negocio, tales como el control y visualización de stock de productos en las sucursales.
- **Administrador (ROLE_ADMIN):** Nivel de seguridad alto. Posee control de acceso total sobre los endpoints sensibles de la aplicación backend, permitiendo la generación de reportes financieros analíticos y tendencias globales de compra.

Justificación de la estrategia seleccionada: Se seleccionó **Spring Security** integrado con **JSON Web Tokens (JWT)**. La justificación técnica radica en que JWT permite una arquitectura **Stateless**. A diferencia del modelo Stateful (sesiones en memoria), JWT no requiere almacenar la sesión en el servidor, lo que mejora drásticamente la escalabilidad del Minimarket Plus para su futura expansión a microservicios. El token es autónomo y está firmado digitalmente, garantizando un acceso rápido y seguro.

2. Guía de configuración y explicación de la implementación.

Para implementar la arquitectura Stateless con JWT, se realizaron los siguientes pasos de configuración:

1. **Integración de Dependencias (pom.xml):** Se añadieron las librerías spring-boot-starter-security para el motor de seguridad, y las librerías de jjwt (jjwt-api, jjwt-impl, jjwt-jackson) para la generación y validación de tokens.
2. **Configuración de Propiedades (application.properties):** Se definió una clave secreta robusta (jwt.secret) y un tiempo de expiración del token (jwt.expiration) para limitar la ventana de vulnerabilidad en caso de robo.
3. **Generación y Validación (Clase JwtUtil):** Se implementó un componente encargado de construir el JWT firmado con el algoritmo **HMAC SHA-256** para validarlo contra los detalles del usuario.
4. **Filtro de Intercepción (JWTAuthFilter):** Se creó un filtro personalizado que extiende de OncePerRequestFilter. Este intercepta el header HTTP Authorization, extrae el Bearer Token, lo valida y, si es correcto, inyecta al usuario en el SecurityContextHolder.
5. **Configuración de Seguridad y Endpoints (SecurityConfig):** Se configuró el SecurityFilterChain para:
 - Deshabilitar el manejo de sesiones en servidor (SessionCreationPolicy.STATELESS).

- Deshabilitar CSRF, ya que las APIs RESTful con JWT no usan cookies de sesión.
- Permitir acceso público a rutas específicas (/api/public/ y /api/auth/).
- Proteger el resto de los endpoints requiriendo autenticación e inyectando nuestro filtro personalizado antes del filtro de validación estándar.

3. Explicación técnica de cómo las configuraciones implementadas protegen los componentes backend frente a las amenazas identificadas.

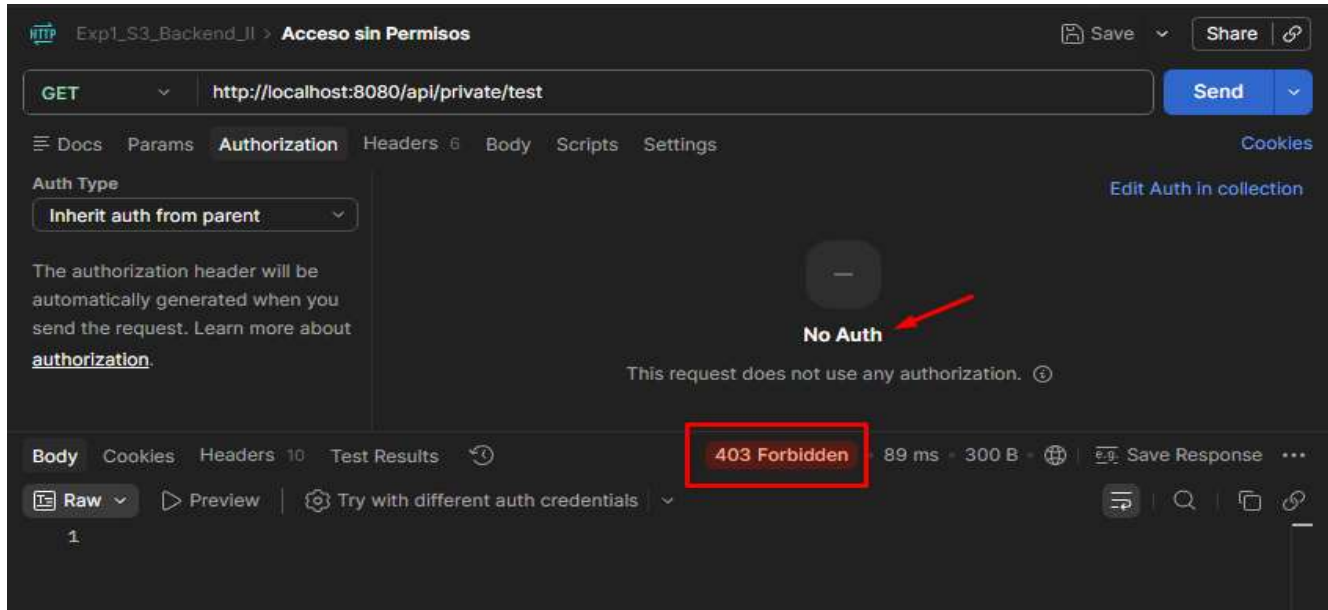
Las configuraciones implementadas protegen activamente los componentes backend del "MINIMARKET PLUS" de la siguiente manera:

- **Mitigación de CSRF (Cross-Site Request Forgery):** Al utilizar JWT y una política Stateless, el backend no depende de las cookies de sesión del navegador enviadas automáticamente. Por lo tanto, el ataque CSRF queda inherentemente neutralizado, justificando el uso de `.csrf().disable()` en la configuración REST.
- **Mitigación de Inyección SQL y XSS:** La integración de Spring Security junto con Spring Data JPA parametriza y resuelve automáticamente las consultas a la base de datos (H2), evitando que sentencias SQL inyectadas en el login (AuthController) sean ejecutadas. Además, el payload JSON está codificado, previniendo inyecciones de scripts.
- **Protección contra Accesos no Autorizados:** Mediante la anotación `@PreAuthorize("hasRole('ADMIN')")` en el controlador, garantizamos que incluso si un atacante consigue un token válido de un cliente normal, el framework bloqueará el acceso al recurso (devolviendo un 403 Forbidden) al no poseer los permisos necesarios.
- **Validación y robustez del Control de Accesos:** La integración de la anotación `@EnableMethodSecurity` junto con expresiones a nivel de método (como `hasRole` y `hasAnyRole`) robustece de forma proactiva la protección contra la elevación

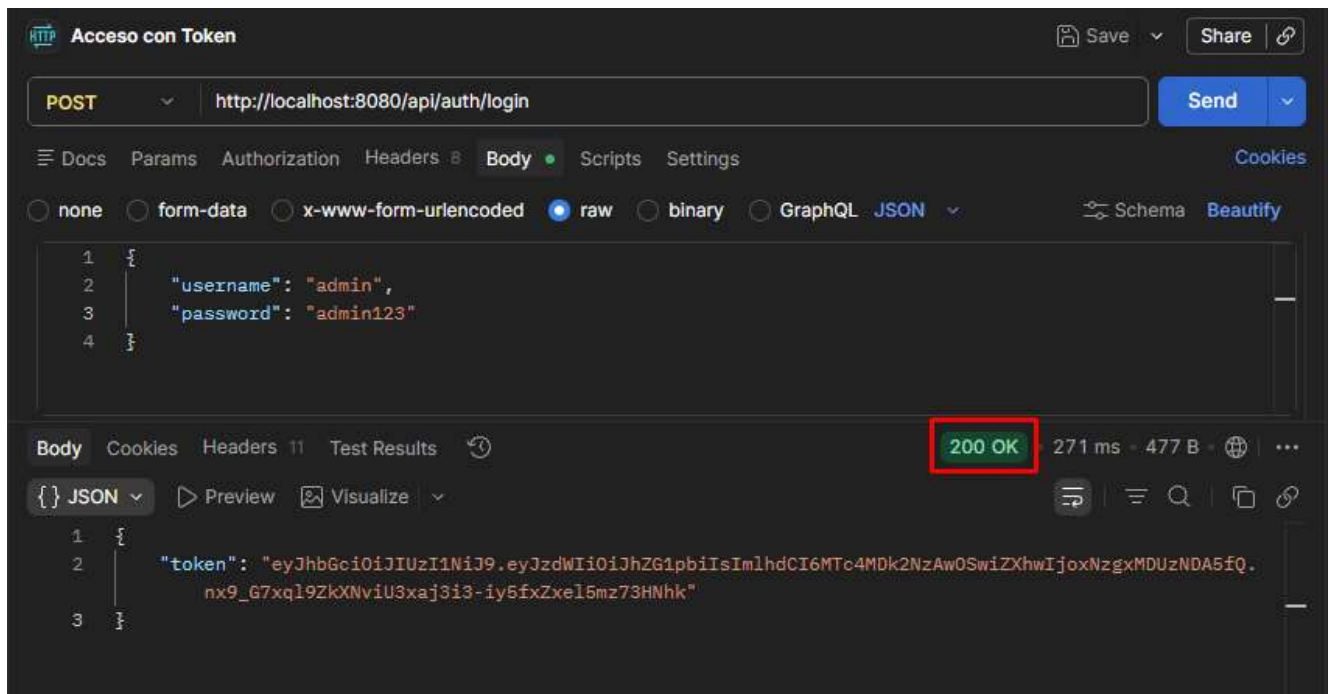
de privilegios de manera drástica. Incluso en el escenario de que una petición legítima de un Cliente sea interceptada y este intente forzar una solicitud REST hacia el endpoint administrativo `/api/admin/reportes`, la arquitectura de filtros verificará los permisos cargados desde el token JWT en el contexto seguro. Al validar que la autoridad del cliente (`ROLE_USER`) no cuenta con las atribuciones requeridas de un administrador (`ROLE_ADMIN`), Spring Security interceptará e invalidará inmediatamente la ejecución del método del controlador, retornando al cliente un código de estado estándar 403 Forbidden, manteniendo así la confidencialidad operativa del negocio.

Evidencias de Pruebas (Casos de Uso):

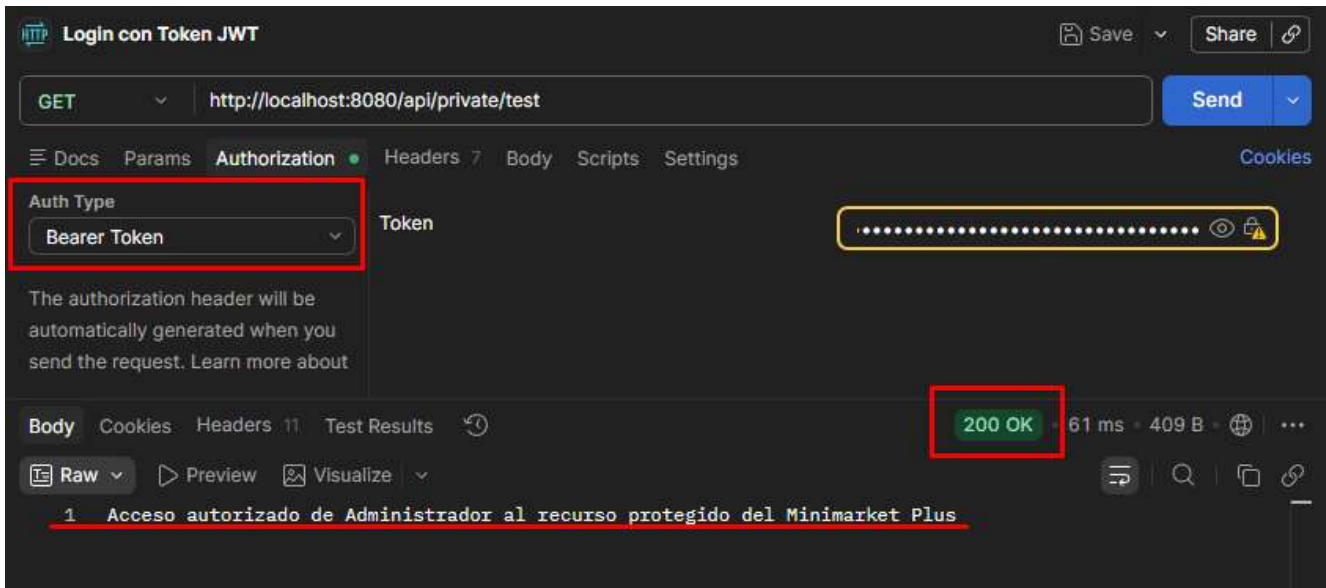
- **Prueba 1: Acceso Denegado (403 Forbidden):**



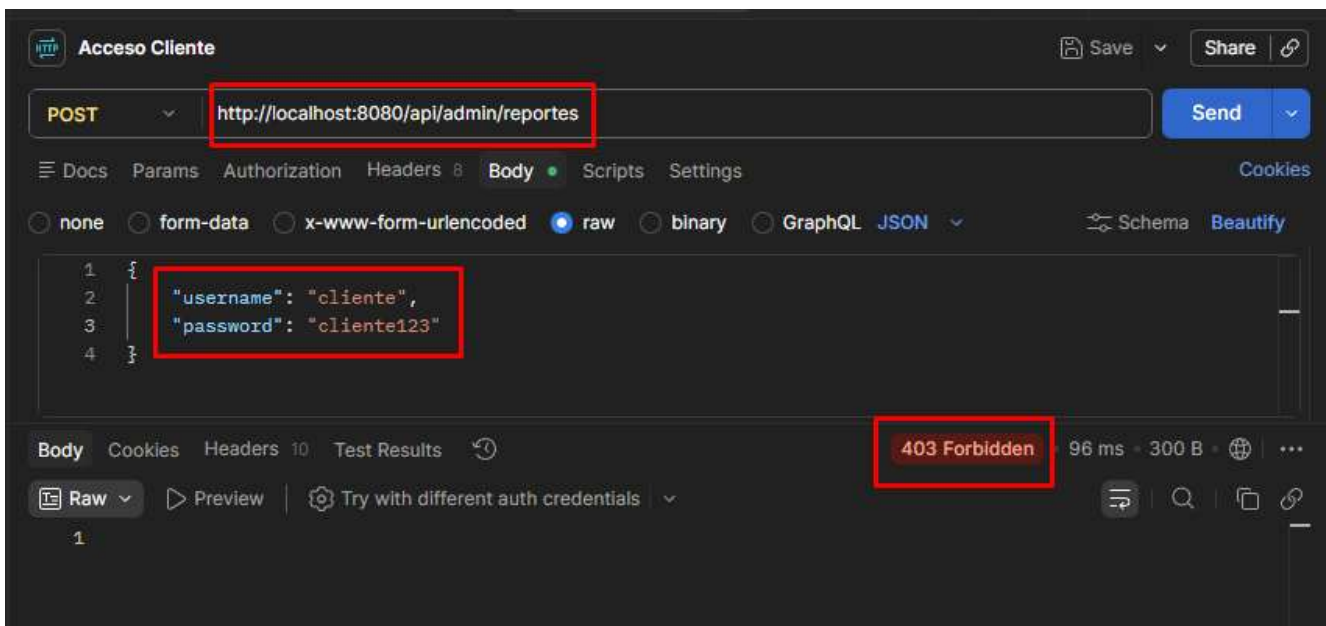
- **Prueba 2: Autenticación Exitosa (Generación de Token):**



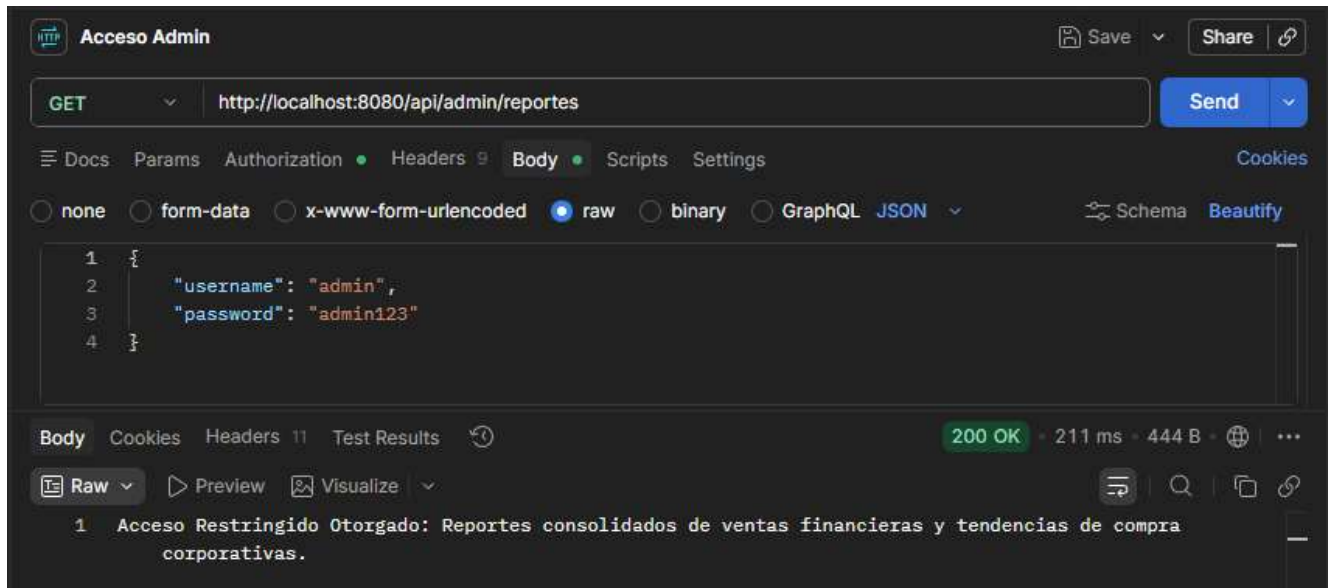
- **Prueba 3:** Autorización por Roles Exitosa (200 OK):



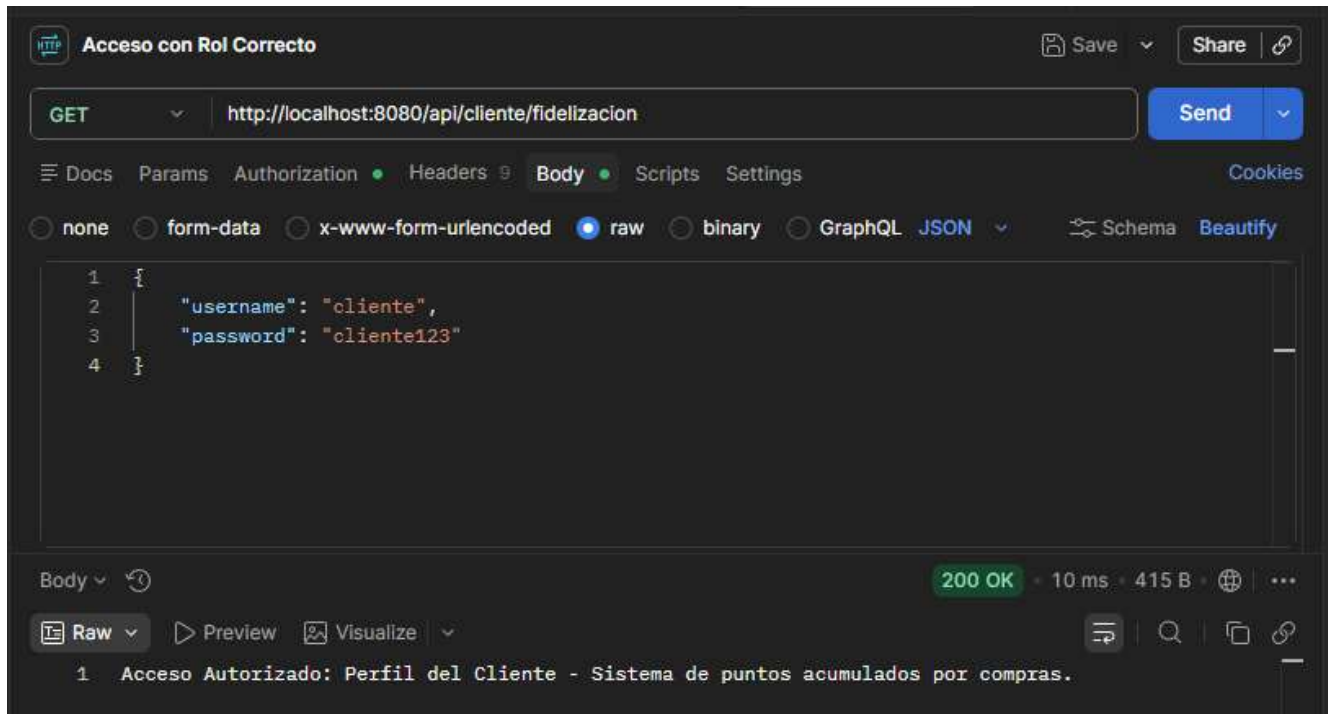
- **Prueba 4:** Intento de acceder a reportes (Restringido a Admin) utilizando credenciales para usuario con Rol Cliente (403 Forbidden):



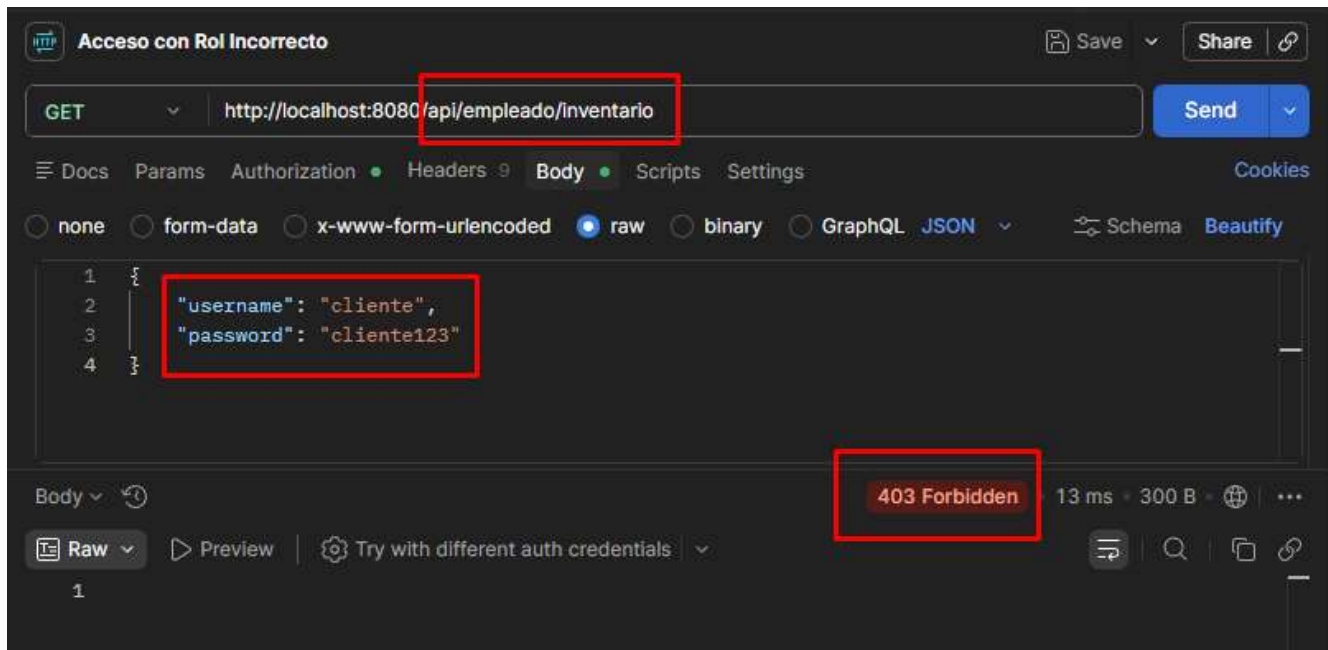
- **Prueba 5:** Intento de acceder a reportes con usuario Admin:



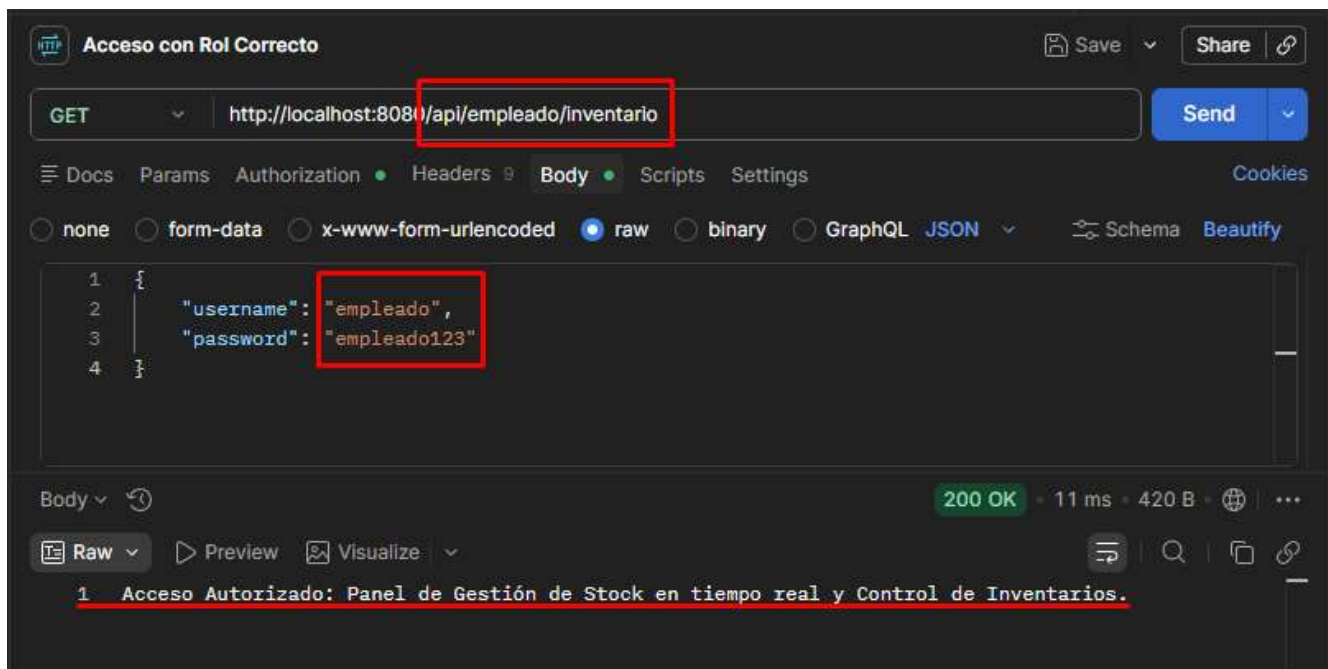
- **Prueba 6:** Acceso a fidelización para clientes con Rol Cliente:



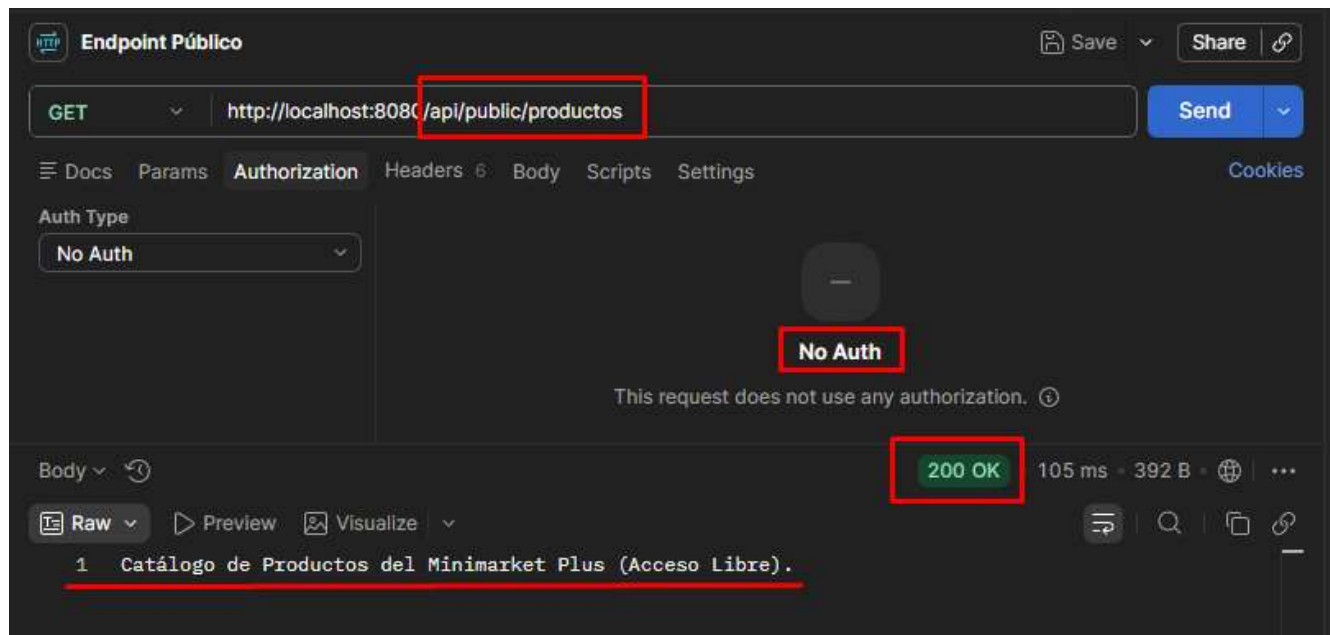
- **Prueba 7:** Intento de acceso a inventario (Restringido a empleados y administrador) utilizando credenciales para usuario con rol Cliente (403 Forbidden):



- **Prueba 8:** Acceso a inventario con Rol empleado:



- **Prueba 9:** Acceso a endpoint publico sin credenciales ni Auth.





Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.